

Linux Commands

Here's the **clean, structured Linux handbook**.

Format: **Command** → what it does



Linux Command Handbook (Beginner Core)



systemd & Services

systemctl

- `systemctl start <service>` → start a service
 - `systemctl stop <service>` → stop a service
 - `systemctl restart <service>` → restart a service
 - `systemctl reload <service>` → reload config without stopping
 - `systemctl status <service>` → show service status
 - `systemctl enable <service>` → start service at boot
 - `systemctl disable <service>` → prevent start at boot
 - `systemctl list-units --type=service` → list running services
 - `systemctl --failed` → show failed services
 - `(sudo) systemctl reboot` → reboot system
 - `(sudo) systemctl poweroff` → shut down system
-

journalctl

- `journalctl` → show all logs
 - `journalctl -u <service>` → logs for specific service
 - `journalctl -f` → follow logs live
 - `journalctl -u <service> -f` → follow logs of a service
 - `journalctl -xe` → latest logs + explanations
 - `journalctl -u <service> -xe` → detailed logs for a service
 - `journalctl -p err -b` → show errors from current boot
-

systemd (concept)

- `.service` file → defines how a service runs

- Located in `/etc/systemd/system/` or `/lib/systemd/system/`
-

File & Directory Management

- `ls` → list files
 - `ls -la` → list all files (including hidden) with details
 - `cd <dir>` → change directory
 - `pwd` → show current directory
 - `mkdir <name>` → create directory
 - `rm <file>` → delete file
 - `rm -r <dir>` → delete directory
 - `cp <src> <dest>` → copy file
 - `mv <src> <dest>` → move or rename file
-

File Viewing & Editing

- `cat <file>` → print file contents
 - `less <file>` → view file with scrolling
 - `head <file>` → show first lines
 - `tail <file>` → show last lines
 - `tail -f <file>` → follow file updates live
 - `nano <file>` → open simple text editor
 - `echo "text" > file` → write text to file
-

Search & Discovery

- `grep "text" <file>` → search text in file
 - `grep -r "text" .` → search recursively
 - `find . -name <file>` → find file by name
 - `which <command>` → show command path
-

Processes & System Monitoring

- `ps aux` → list all processes
- `top` → real-time system usage
- `htop` → improved process viewer (if installed)
- `kill <pid>` → stop process by ID

Package Management

Debian/Ubuntu:

- `sudo apt update` → update package list
- `sudo apt install <pkg>` → install package
- `sudo apt remove <pkg>` → remove package

Fedora:

- `sudo dnf install <pkg>` → install package

Permissions & Ownership

- `chmod +x <file>` → make file executable
- `chmod 755 <file>` → set permissions
- `chown user:<file>` → change file owner

Networking

- `ping <host>` → test connectivity
- `curl <url>` → fetch data from URL
- `wget <url>` → download file
- `ip a` → show IP addresses

Terminal Productivity

- `history` → show command history
- `clear` → clear terminal screen
- `sudo !!` → rerun last command with sudo

Service Example (nginx)

- `systemctl start nginx` → start nginx
- `systemctl status nginx` → check nginx
- `journalctl -u nginx` → view nginx logs

⚡ Core Set (Must Memorize First)

If you only remember a few:

- `ls` → list files
- `cd` → navigate
- `pwd` → current location
- `cp / mv / rm` → manage files
- `cat / less` → read files
- `grep` → search
- `systemctl` → manage services
- `journalctl` → view logs

🧠 Final Mental Model

- **Files** → everything stored 📁
- **Processes** → everything running ⚙️
- **Services (.service)** → managed background apps 🔄
- **Logs (journalctl)** → what happened 📖

To find **running Python processes with grep**, you combine it with `ps`:

🔍 Basic Command

```
ps aux | grep python
```

👉 What it does:

- `ps aux` → lists all running processes
- `|` → sends output to next command
- `grep python` → filters only lines containing "python"

⚠️ Small Issue (Important)

This command also shows the **grep process itself**.

Example output might include:

```
user 1234 ... python app.py
user 5678 ... grep python ← not useful
```

✅ Better Version (Clean Output)

```
ps aux | grep '[p]ython'
```

👉 Why this works:

- The pattern `[p]ython` matches "python"
- But **does NOT match the grep command itself**

🎯 Even Better (Recommended)

```
pgrep -a python
```

👉 What it does:

- Shows only Python processes
- Cleaner and faster than `ps + grep`

🧠 Summary

- `ps aux | grep python` → basic way
- `ps aux | grep '[p]ython'` → avoids self-match
- `pgrep -a python` → best and cleanest

💡 Real Example Use

If you're running:

```
python app.py
```

Then:

```
ps aux | grep python
```

👉 will show your script + PID (process ID)

To monitor a Python process in real time, you have a few good options depending on what you want to see 👉

🔴 1. Using top (built-in, simple)

```
top
```

👉 Then press:

Shift + L

Type:

```
python
```

What you get:

- Live CPU usage ⚡
 - Memory usage 🧠
 - Process IDs (PID)
-

🟢 2. Using htop (best interactive view)

```
htop
```

👉 Then press:

F3

Type:

python

Why it's better:

- Easier to read
 - Colorful UI
 - Can scroll, filter, and kill processes easily
-

3. Using watch + ps (simple live refresh)

```
watch -n 1 "ps aux | grep '[p]ython'"
```

What it does:

- Updates every **1 second**
 - Shows only Python processes
-

4. Monitor a specific process (by PID)

First get PID:

```
pgrep -a python
```

Then monitor:

```
top -p <PID>
```

👉 Example:

```
top -p 1234
```

5. If you want logs (for services)

If Python app runs as a service:

```
journalctl -u <service> -f
```

👉 Real-time logs output

🧠 When to use what

- Quick check → ps + grep
 - Live system view → top
 - Best experience → htop
 - Script-friendly → watch
 - Logs/debugging → journalctl -f
-

⚡ Pro Tip

If CPU usage spikes:

```
top
```

Then press:

Shift + P

👉 sorts by CPU usage (very useful)

🌐 Linux Networking (Copy / Move Between Systems)

Format: Command → what it does

🔄 scp (Secure Copy over SSH)

scp file user@host:/path → copy file to remote server

👉 Example:

```
scp app.py user@192.168.1.10:/home/user/
```


👉 Use case:

Upload a script or config file to a server quickly

```
scp user@host:/path/file . → copy file from remote
```

👉 Example:

```
scp user@192.168.1.10:/var/log/app.log .
```

👉 Use case:

Download logs for debugging

🔄 rsync (Efficient Sync & Transfer)

```
rsync -avz dir user@host:/path → sync directory
```

👉 Example:

```
rsync -avz project/ user@192.168.1.10:/home/user/project/
```

👉 Use case:

Deploy code updates (only changed files transfer)

```
rsync -av --delete dir user@host:/path → mirror directories
```

👉 Example:

```
rsync -av --delete backup/ user@server:/data/backup/
```

👉 Use case:

Keep remote backup identical to local

📦 sftp (Interactive File Transfer over SSH)

```
sftp user@host → connect
```

👉 Example:

```
sftp user@192.168.1.10
```

put file → upload

👉 Example inside session:

```
put app.py
```

👉 Use case:

Manually upload/download files when browsing remote directories

🚀 ssh + tar (Fast Bulk Transfer)

`tar czf - dir | ssh user@host "tar xzf -" → send directory`

👉 Example:

`tar czf - project/ | ssh user@192.168.1.10 "tar xzf -"`

👉 Use case:

Transfer thousands of small files quickly

`ssh user@host "tar czf - /data" | tar xzf - → receive directory`

👉 Example:

`ssh user@server "tar czf - /var/www" | tar xzf -`

👉 Use case:

Pull full website or dataset

 **wget (Download from URL)**

`wget http://host/file → download file`

👉 Example:

`wget https://example.com/file.zip`

👉 Use case:

Download packages, datasets, or installers

`wget -c http://host/file → resume download`

👉 Example:

`wget -c https://example.com/large.iso`

👉 Use case:

Resume interrupted large downloads

 **curl (Flexible data transfer)**

`curl -O http://host/file → download file`

👉 Example:

`curl -O https://example.com/data.json`

👉 Use case:

Fetch API data or files in scripts

`curl -o name http://host/file → custom filename`

👉 Example:

`curl -o output.json https://api.site/data`

👉 Use case:

Save API responses with specific names

🔗 **sshfs (Mount remote directory)**

```
sshfs user@host:/remote/path /local/mount → mount
```

👉 Example:

```
sshfs user@192.168.1.10:/home/user /mnt/remote
```

👉 Use case:

Edit remote files like they're local

```
fusermount -u /local/mount → unmount
```

👉 Example:

```
fusermount -u /mnt/remote
```

📡 **netcat (nc) – raw transfer**

```
nc -l -p 1234 > file → receive file
```

👉 Example (receiver):

```
nc -l -p 1234 > backup.tar
```

```
nc host 1234 < file → send file
```

👉 Example (sender):

```
nc 192.168.1.10 1234 < backup.tar
```

👉 Use case:

Quick transfer in trusted local networks (no SSH needed)

🧠 **When to use what**

- scp → quick one-time copy
- rsync → backups, deployments, syncing
- sftp → manual file browsing
- tar + ssh → huge directory transfers
- wget/curl → internet downloads / APIs
- sshfs → remote editing
- netcat → fast local transfers (advanced)

⚡ Real-World Scenarios

- Deploy app → `rsync project/ server:/app/`
 - Grab logs → `scp server:/var/log/app.log .`
 - Backup server → `rsync -delete`
 - Edit remote config → `sshfs`
 - Download dataset → `wget`
 - API automation → `curl`
-